Buddha Institute of Technology

Department of COMPUTER SCIENCE & ALLIED

PROGRAM: AIML-DS

(Affiliated to Dr. A. P. J. Abdul Kalam Technical University, Lucknow)

Computer Networks

LAB MANUAL

VI semester, Course Code: BCS653

[As per the Choice Based Credit System Scheme]
Scheme: 2025-26

Version 1.1
w.e.f. 29th Jan, 2024

Proposed by:
Mr. Shailesh Kumar Patel

Experiment # 1

OBJECT: To learn handling and configuration of networking hardware like RJ-45 connector CAT- 6 cable and crimping tools.

THEORY:

RJ-45: RJ45 cable is used for connect the ALL HMI and engineer station through a switch to communicate each other. It is used to download the any modification and which is made in graphics in engineering station. RJ45 cable also used for communicate the printer with computer. There are four pairs of wires in an Ethernet cable, and an Ethernet connector (8P8C) has eight pin slots. Each pin is identified by a number, starting from left to right, with the clip facing away from you.

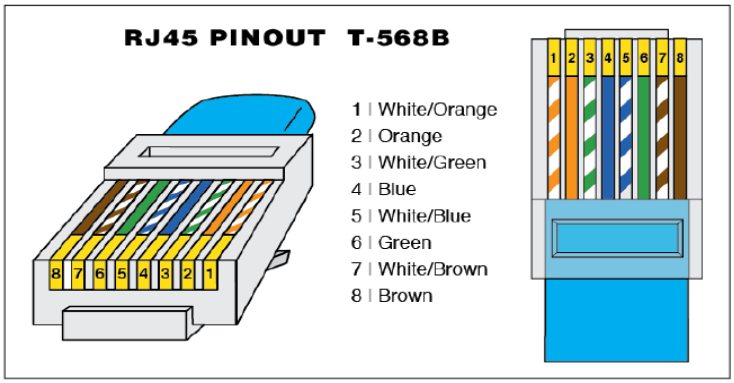


Fig 01: RJ-45 Connector

CAT-6 CABLE: Category 6 cable, also commonly known as network, LAN or Ethernet data cable, is a 4 twisted pair sheathed copper wire cable that can support data transfer rates of up to 1 gigabits (1,000 megabits). This higher bandwidth allows for quick transferred of large files in an office network.

It is compatible with fast Ethernet 10BASE-T, 100BASE-TX and Gigabit networks, and is backwards compatible with previous iterations, such as Cat5/5E and Cat3.

Either end usually has an eight-position eight connector (8P8C) RJ45 jack which is used to connect two devices together with the Cat6 cable. It's important to use jacks that meet the Cat6 rating in order to get the full performance it delivers.

As with all Ethernet cables, it can be identified by the labeling on the outer sheath.

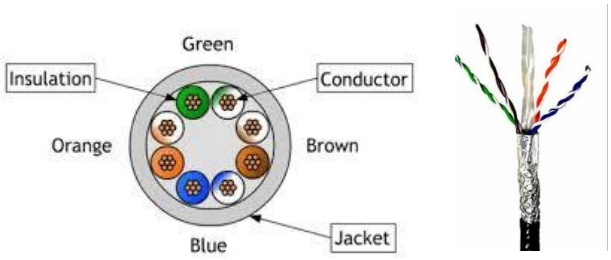


Fig 02: CAT 6 Cable

CRIMPING TOOL: Crimpers are tools used to make cold weld joints between two wires or a wire and a connector, such as lugs. Ideally, the electrical and mechanical properties of the weld joint are as strong as the parent materials. Crimping tools are sized according to the wire gauges (using AWG - American Wire Gauge) they can accept. Some come with interchangeable die heads that allow for a wider range of wire sizes and connectors.

How to use: First you will need to strip the length of wire that you want to crimp. Then, attach the connector. For crimping tools with interchangeable dies, you will need to select the right die head for the connector by matching wire gauge ratings. For die less crimpers, you will need to match to the proper groove. Finally, apply pressure, take out the newly crimped connector, and give a few tugs to make sure you have a solid and secure connection.

Fig 03: Crimping Tool

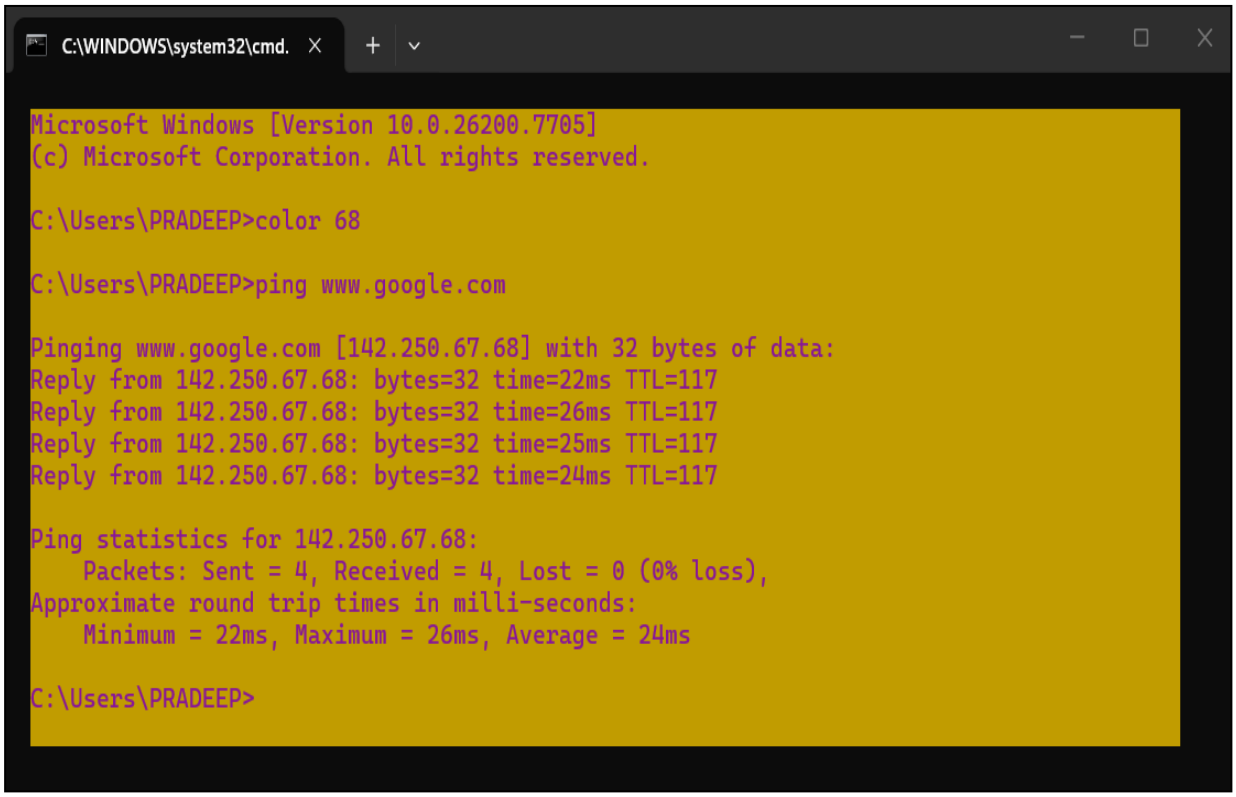
Experiment # 2

OBJECT - Running and using services/commands like ping, traceroute, nslookup etc.

PING: PACKET INTERNET GOPHER

- The ping command is used to determine the ability of a user's computer to reach a destination computer.
- The main purpose of using this command is to verify if the computer can connect over the network to another computer or network device.
- The /usr/sbin/ping command sends an Internet Control Message Protocol (ICMP) ECHO_REQUEST to obtain an ICMP ECHO_RESPONSE from a host or gateway. The ping command is useful for:
    i. Determining the status of the network and various foreign hosts.
    ii. Tracking and isolating hardware and software problems.
    iii. Testing, measuring, and managing networks.

Ping (Packet Internet Groper) is a method for determining communication latency between two networks. Simply put, ping is a method of determining latency or the amount of time it takes for data to travel between two devices or across a network. As communication latency decreases, communication effectiveness improves.



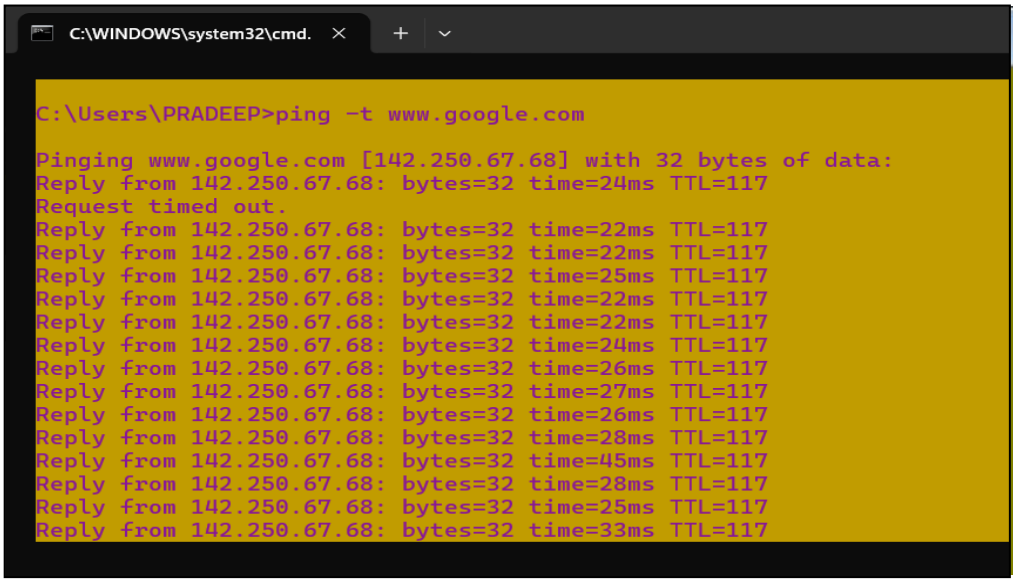


Fig. 01: Ping command

TRACEROUTE:

Traceroute is a networking tool used to trace the path that a packet of data takes from the source to its destination. It works by sending a series of packets with varying Time-to-Live (TTL) values and then analyzing the ICMP error messages that are returned to determine the route taken by the data. Traceroute provides valuable information about the number of hops and the latency between each node, helping to identify network connectivity issues and troubleshoot problems in the network. It is a useful command for network administrators and IT professionals to diagnose and resolve network problems.

Example:-

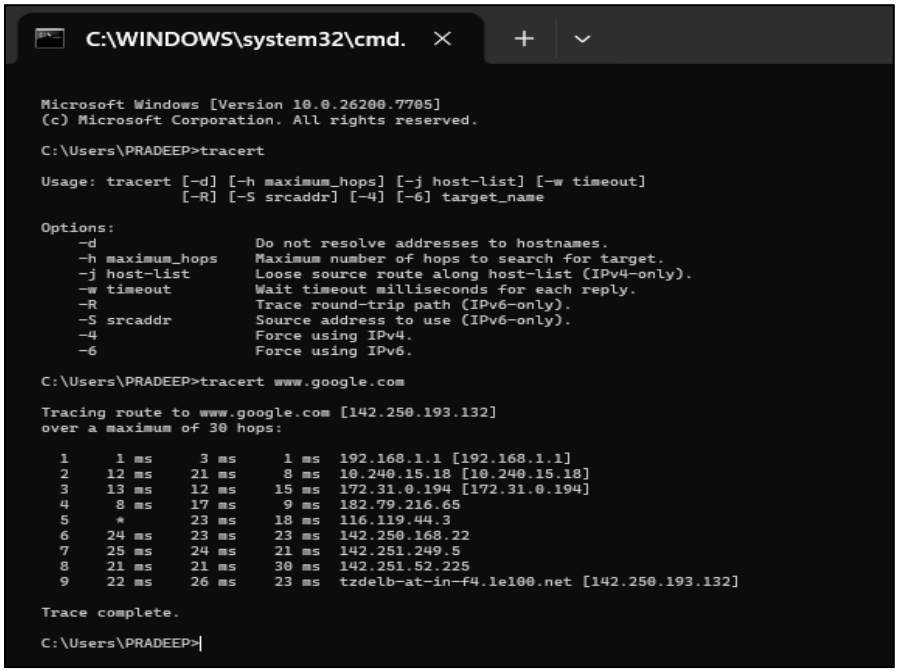


Fig. 02: Traceroute command

NSLOOKUP:

The lookup command is a helpful feature in many programming languages and databases that allows users to search for specific data or information within a given dataset. It essentially works by matching a specified key or value to an entry within a table or list, and returning the corresponding result.

In databases, the lookup command is often used to quickly retrieve records based on a primary key, such as a customer ID or product code. This can be especially useful when dealing with large datasets, as it allows for efficient data retrieval without the need to scan through each record individually.

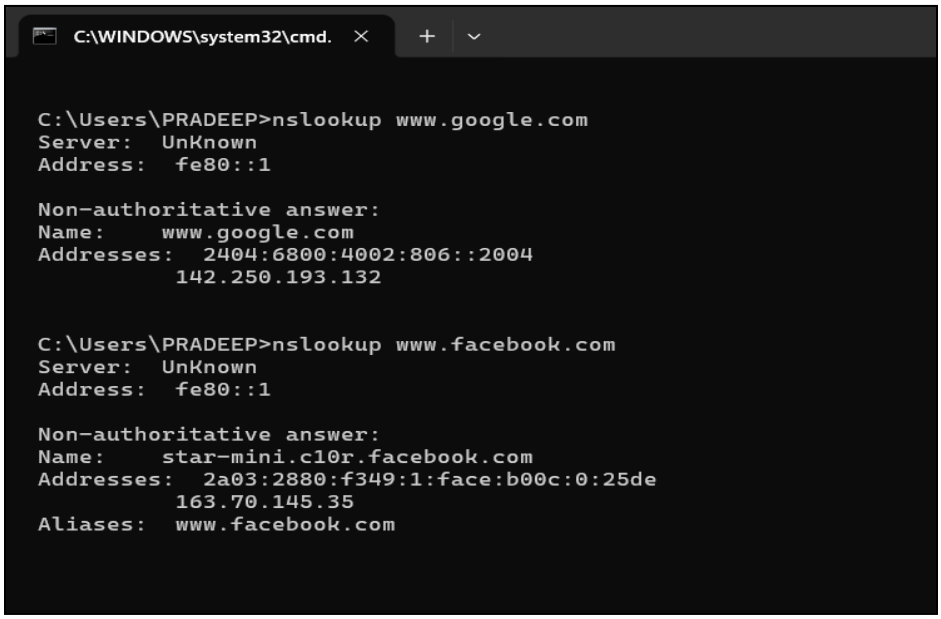


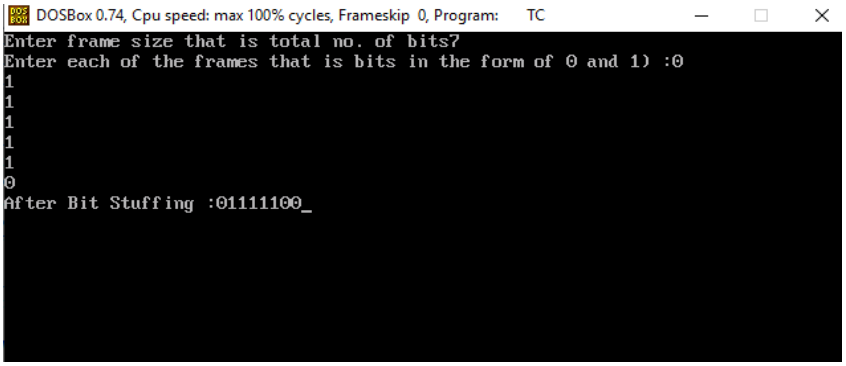
Fig 03: Nslookup command

Experiment # 3

OBJECT-Write a C program to implement data link layer framing method (Bit Stuffing).

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    int a[20],b[30],i,j,k,count,n;
    clrscr();
    printf("Enter frame size that is total no. of bits");
    scanf("%d",&n);
    printf("Enter each of the frames that is bits in the form of 0 and
1) :");
    for(i=0; i<n; i++)
        scanf("%d",&a[i]);
    i=0;
    count=1;
    j=0;
    while(i<n)
    {
        if(a[i]==1)
        {
            b[j]=a[i];
            for(k=i+1; a[k]==1 && k<n && count<5; k++)
            {
                j++;
                b[j]=a[k];
                count++;
                if(count==5)
                {
                    j++;
                    b[j]=0;
                }
                i=k;
            }
        }
        else
        {
            b[j]=a[i];
        }
        i++;
        j++;
    }
    printf("After Bit Stuffing :");
```

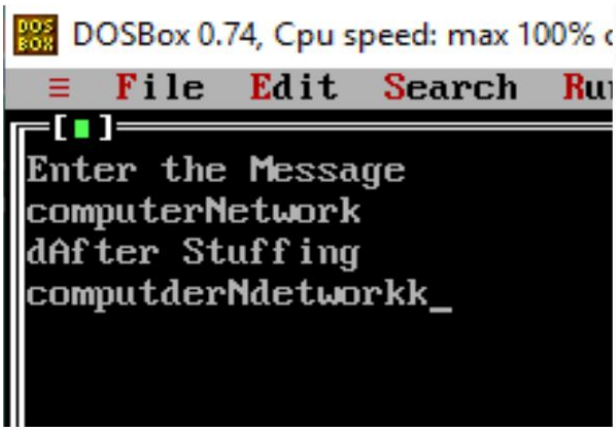
```
    for(i=0; i<j; i++)
        printf("%d",b[i]);
    getch();
}
```



Experiment # 4

Object: Write a C program to implement data link layer framing method (Character Stuffing).

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
      char msgIn[999], msgOut[999];
      int i, j=0, len;
      clrscr();
      printf("Enter the Message");
      gets(msgIn);
      len=strlen(msgIn);
      for(i=0;i<len;i++)
      {
            if(msgIn[i]=='s' || msgIn[i]=='d' || msgIn[i]=='e')
                  msgOut[j++]='d';
                  msgOut[j++]=msgIn[i];
      }
      printf("After Stuffing \n");
      printf("%s",msgOut);
      getch();
}
```



Experiment # 5

Object: Write a program in C/C++/Java to find the IP address of a given host.

```
import java.net.*;
public class Demo
{
  public static void main(String[] args)
   {
    try
    {
      InetAddress my_address = InetAddress.getLocalHost();
      System.out.println("The IP address is : " +
my_address.getHostAddress());
      System.out.println("The host name is : " +
my_address.getHostName());
    }
      catch (UnknownHostException e)
    {
      System.out.println( "Couldn't find the local address.");
    }
  }
}
```

Experiment # 6

Object: Study of socket programming and client-server model.

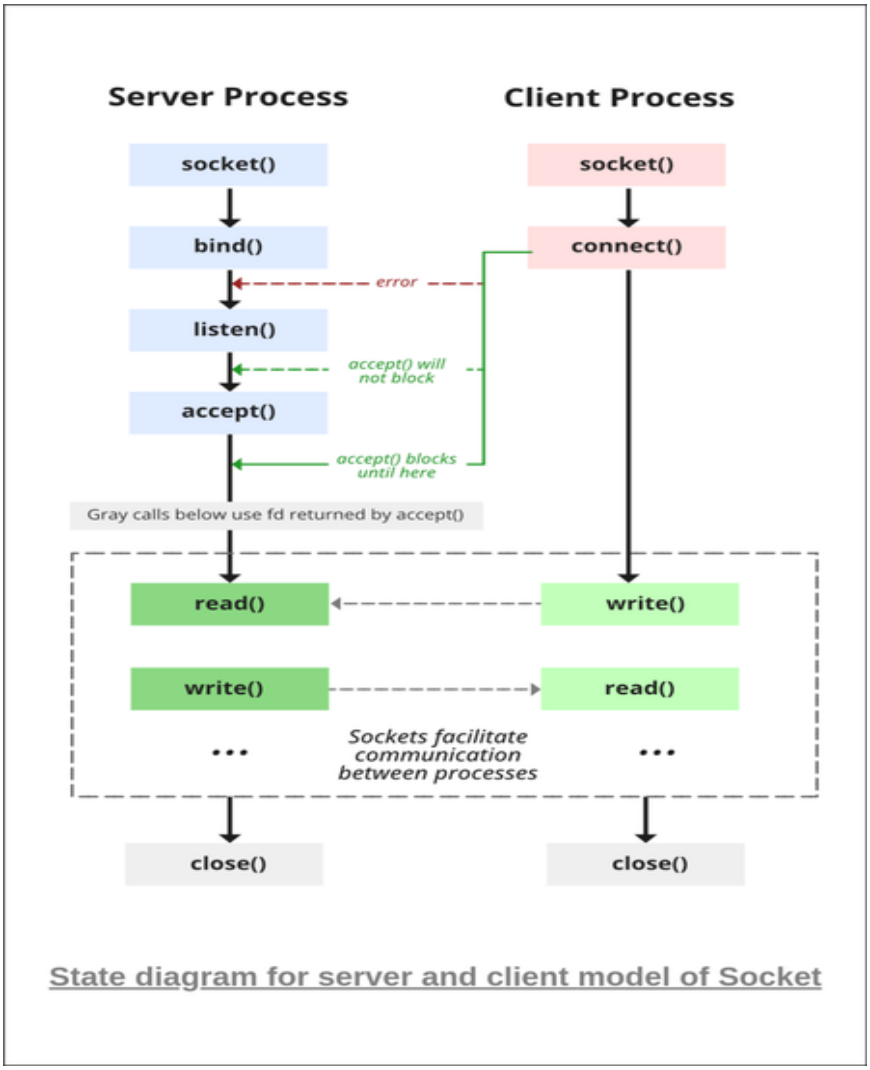
A socket is one endpoint of a two way communication link between two programs running on the network. The socket mechanism provides a means of inter-process communication (IPC) by establishing named contact points between which the communication takes place.

Like 'Pipe' is used to create pipes and sockets is created using 'socket' system call. Each socket has a specific address. This address is composed of an IP address and a port number.

Sockets are generally employed in client server applications. The server creates a socket, attaches it to a network port addresses then waits for the client to contact it.

Socket programming is a way of connecting two nodes on a network to communicate with each other. One socket (node) listens on a particular port at an IP, while the other socket reaches out to the other to form a connection. The server forms the listener socket while the client reaches out to the server.

ALGORITHM:



State diagram for server and client model of Socket

Server:

Step1:Create a server socket and bind it to port.
Step 2:Listen for new connection and when a connection arrives, accept it.
Step 3:Send server's date and time to the client.
Step 4:Read client's IP address sent by the client.
Step 5:Display the client details.
Step 6:Repeat steps 2-5 until the server is terminated.
Step 7:Close the server socket.

Client:

Step1:Create a client socket and connect it to the server's port number.
Step2:Retrieve its own IP address using built-in function.
Step3:Send its address to the server.
Step4:Display the date & time sent by the server.
Step5:Close the client socket

Stages for Server:

The server is created using the following steps:

1. Socket Creation

int sockfd = socket(domain, type, protocol)

sockfd: socket descriptor, an integer (like a file handle)
domain: integer, specifies communication domain.
type: communication type
SOCK_STREAM: TCP(reliable, connection-oriented)
SOCK_DGRAM: UDP(unreliable, connectionless)
protocol: Protocol value for Internet Protocol(IP)

2. Setsockopt

This helps in manipulating options for the socket referred by the file descriptor sockfd. This is completely optional, but it helps in reuse of address and port. Prevents error such as: "address already in use".

int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_toptlen);

3. Bind

After the creation of the socket, the bind function binds the socket to the address and port number specified in addr(custom data structure). In the example code, we bind the server to the localhost.

int bind(int sockfd, const struct sockaddr *addr, socklen_taddrlen);

4. Listen

It puts the server socket in a passive mode, where it waits for the client to approach the server to make a connection. The backlog, defines the maximum length to which the queue of pending connections for sockfd may grow. If a connection request arrives when the queue is full, the client may receive an error with an indication of ECONNREFUSED.

int listen(int sockfd, int backlog);

5. Accept

It extracts the first connection request on the queue of pending connections for the listening socket, sockfd, creates a new connected socket, and returns a new file descriptor referring to that socket. At this point, the connection is established between client and server, and they are ready to transfer data.

int new_socket= accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);

Stages for Client:

1. Socket connection

Exactly the same as that of server's socket creation

2. Connect

The connect() system call connects the socket referred to by the file descriptor sockfd to the address specified by addr. Server's address and port is specified in addr.

int connect(int sockfd, const struct sockaddr *addr, socklen_taddrlen);

Experiment # 7

Object: Perform a case study about the different routing algorithms to select the network path with its optimum and economical during data transfer. i. Link State routing ii. Flooding iii. Distance vector.

i.Link State Routing

Routing is the process of selecting best paths in a network. In the past, the term routing was also used to mean forwarding network traffic among networks. However, this latter function is much better described as simply forwarding. Routing is performed for many kinds of networks, including the telephone network (circuit switching), electronic data networks (such as the Internet), and transportation networks. This article is concerned primarily with routing in electronic data networks using packet switching technology.

In packet switching networks, routing directs packet forwarding (the transit of logically addressed network packets from their source toward their ultimate destination) through intermediate nodes. Intermediate nodes are typically network hardware devices such as routers, bridges, gateways, firewalls, or switches. General-purpose computers can also forward packets and perform routing, though they are not specialized hardware and may suffer from limited performance. The routing process usually directs forwarding on the basis of routing tables which maintain a record of the routes to various network destinations. Thus, constructing routing tables, which are held in the router's memory, is very important for efficient routing. Most routing algorithms use only one network path at a time. Multipath routing techniques enable the use of multiple alternative paths.

In case of overlapping/equal routes, the following elements are considered in order to decide which routes get installed into the routing table (sorted by priority):

1. Prefix-Length: where longer subnet masks are preferred (independent of whether it is within a routing protocol or over different routing protocol)

2. Metric: where a lower metric/cost is preferred (only valid within one and the same routing protocol)

3. Administrative distance: where a lower distance is preferred (only valid between different routing protocols)

Routing, in a more narrow sense of the term, is often contrasted with bridging in its assumption that network addresses are structured and that similar addresses imply proximity within the network. Structured addresses allow a single routing table entry to represent the route to a group of devices. In large networks, structured addressing (routing, in the narrow sense) outperforms unstructured addressing (bridging). Routing has become the dominant form of addressing on the Internet. Bridging is still widely used within localized environments.

ii.Flooding

Flooding is a simple routing algorithm in which every incoming packet is sent through every outgoing link except the one it arrived on. Flooding is used in bridging and in systems such as

Usenet and peer-to-peer file sharing and as part of some routing protocols, including OSPF, DVMRP, and those used in ad-hoc wireless networks. There are generally two types of flooding available, Uncontrolled Flooding and Controlled Flooding. Uncontrolled Flooding is the fatal law of flooding. All nodes have neighbors and route packets indefinitely. More than two neighbors create a broadcast storm.

Controlled Flooding has its own two algorithms to make it reliable, SNCF (Sequence Number Controlled Flooding) and RPF (Reverse Path Flooding). In SNCF, the node attaches its own address and sequence number to the packet, since every node has a memory of addresses and sequence numbers. If it receives a packet in memory, it drops it immediately while in RPF, the node will only send the packet forward. If it is received from the next node, it sends it back to the sender.

Algorithm

There are several variants of flooding algorithm. Most work roughly as follows:

- Each node acts as both a transmitter and a receiver.
- Each node tries to forward every message to every one of its neighbors except the source node.

This results in every message eventually being delivered to all reachable parts of the network.

Algorithms may need to be more complex than this, since, in some case, precautions have to be taken to avoid wasted duplicate deliveries and infinite loops, and to allow messages to eventually expire from the system. A variant of flooding called selective flooding partially addresses these issues by only sending packets to routers in the same direction. In selective flooding the routers don't send every incoming packet on every line but only on those lines which are going approximately in the right direction.

Advantages

- If a packet can be delivered, it will (probably multiple times).
- Since flooding naturally utilizes every path through the network, it will also use the shortest path.
- This algorithm is very simple to implement.

Disadvantages

- Flooding can be costly in terms of wasted bandwidth. While a message may only have one destination it has to be sent to every host. In the case of a ping flood or a denial of service attack, it can be harmful to the reliability of a computer network.
- Messages can become duplicated in the network further increasing the load on the networks bandwidth as well as requiring an increase in processing complexity to disregard duplicate messages.
- Duplicate packets may circulate forever, unless certain precautions are taken.
- Use a hop count or a time to live count and include it with each packet. This value should consider the number of nodes that a packet may have to pass through on the way to its destination.

- Have each node keep track of every packet seen and only forward each packet once Enforce a network topology without loops

iii. Distance vector

In computer communication theory relating to packet-switched networks, a distance vector routing protocol is one of the two major classes of routing protocols, the other major class being the link-state protocol. Distance-vector routing protocols use the Bellman–Ford algorithm, Ford–Fulkerson algorithm, or DUAL FSM (in the case of Cisco Systems' protocols) to calculate paths.

A distance-vector routing protocol requires that a router informs its neighbors of topology changes periodically. Compared to link-state protocols, which require a router to inform all the nodes in a network of topology changes, distance-vector routing protocols have less computational complexity and message overhead.

The term distance vector refers to the fact that the protocol manipulates vectors (arrays) of instances to other nodes in the network. The vector distance algorithm was the original ARPANET routing algorithm and was also used in the internet under the name of RIP (Routing Information Protocol). Examples of distance-vector routing protocols include RIPv1 and RIPv2 and IGRP.

Method

Routers using distance-vector protocol do not have knowledge of the entire path to a destination. Instead they use two methods:

- Direction in which router or exit interface a packet should be forwarded.
- Distance from its destination

Distance-vector protocols are based on calculating the direction and distance to any link in a network. "Direction" usually means the next hop address and the exit interface. "Distance" is a measure of the cost to reach a certain node.The least cost route between any two nodes is the route with minimum distance. Each node maintains a vector (table) of minimum distance to every node. The cost of reaching a destination is calculated using various route metrics. RIP uses the hop count of the destination whereas IGRP considers other information such as node delay and available bandwidth.

Updates are performed periodically in a distance-vector protocol where all or part of arouter's routing table is sent to all its neighbors that are configured to use the same distancevectorrouting protocol. RIP supports cross-platform distance vector routing whereas IGRP is aCisco Systems proprietary distance vector routing protocol. Once a router has this information itis able to amend its own routing table to reflect the changes and then inform its neighbors of thechanges. This process has been described as routing by rumor's because routers are relying onthe information they receive from other routers and cannot determine if the information isactually valid and true. There are a number of features which can be used to help with instability
and inaccurate routing information.

EGP and BGP are not pure distance-vector routing protocols because a distance-vectorprotocol calculates routes based only on link costs whereas in BGP, for example, the local routepreference value takes priority over the link cost.

Count-to-infinity problem

The Bellman–Ford algorithm does not prevent routing loops from happening and suffersfrom the count-to-infinity problem. The core of the count-to-infinity problem is that if A tells Bthat it has a path somewhere, there is no way for B to know if the path has B as a part of it. Tosee the problem clearly, imagine a subnet connected like A–B–C–D–E–F, and let the metricbetween the routers be "number of jumps".

Now suppose that A is taken offline. In the vector-update-process B notices that the routeto A, which was distance 1, is down – B does not receive the vector update from A. The problemis, B also gets an update from C, and C is still not aware of the fact that A is down – so it tells Bthat A is only two jumps from C (C to B to A), which is false. This slowly propagates throughthe network until it reaches infinity (in which case the algorithm corrects itself, due to the relaxation property of Bellman–Ford).

RESULT

Thus, the case study about the different routing algorithms to select the network path withits optimum and economical during data transfer was performed.

Experiment # 8

Object: Program using TCP Sockets Date and Time Server.

tcpdateserver.java

```
import java.net.*;
import java.io.*;
import java.util.*;
class tcpdateserver
{
     public static void main(String args[])
     {
          ServerSocket ss=null;
          Socket cs;
          PrintStreamps;
          BufferedReader dis;
          String inet;
          try
          {
               ss= new ServerSocket(4444);
               System.out.println("Press Ctrl+C to quit");
               while(true)
               {
                    cs=ss.accept();
                    ps=new PrintStream(cs.getOutputStream());
                    Date d=new Date();
                    ps.println(d);
                    dis=new BufferedReader(new
InputStreamReader(cs.getInputStream()));
                    inet=dis.readLine();
                    System.out.println("Client System/IP
address is:"+inet);
                    ps.close();
                    dis.close();
               }
          }
                    catch(IOException e)
                    {
```

```
                    System.out.println("The exception
is:"+e);
                }
     }


}
```

tcpdateclient.java

```
import java.net.*;
import java.io.*;
class tcpdateclient{
          public static void main(String args[])
          {
               Socket soc;
               BufferedReader dis;
               String sdate;
               PrintStreamps;
               try
               {
                    InetAddressia =
InetAddress.getLocalHost();
                    if(args.length==0)
                    {
                         soc=new
Socket(InetAddress.getLocalHost(),4444);
                    }
                    else
                    {
                     soc=new
Socket(InetAddress.getByName(args[0]),4444);
                    }
                    dis=new BufferedReader(new
InputStreamReader(soc.getInputStream()));
                    sdate=dis.readLine();
                    System.out.println("The date/time on
server is:"+sdate);
                    ps=new PrintStream(soc.getOutputStream());
```

```
                    ps.println(ia);
                    ps.close();
               }
                    catch(IOException e)
                    {
                         System.out.println("THE EXCEPTION is:
"+e);
                    }
               }
          }
```

Experiment # 9

Object: Implementation of Client-Server Communication using TCP.

server.java

```
import java.io.*;
import java.net.*;
class server{
     public static void main(String args[]){
          String data ="Network Lab";
          try{
               ServerSocketsrvr=new ServerSocket(1234);
               Socket skt= srvr.accept();
               System.out.println("Server has
connected!!!\n");
               PrintWriter out=new
PrintWriter(skt.getOutputStream(), true);
               System.out.print("Sending String:
"+data+"\n");
               out.print(data);
               out.close();
               skt.close();
               srvr.close();
          }
          catch(Exception e){
               System.out.println("It didn't work");
          }
     }
}
```

client.java

```
import java.io.*;
import java.net.*;
class client{
     public static void main(String args[]){
          try{
```

```
                Socket skt= new Socket("localhost",1234);
                BufferedReader in=new BufferedReader(new
    InputStreamReader(skt.getInputStream()));
                System.out.println("Received string: ");
                while(!in.ready()){}
                System.out.println(in.readLine());
                System.out.println("\n");
                in.close();
           }
           catch(Exception e){
                System.out.print("Error");
           }
      }
    }
```

Experiment # 10

Object: Implementation of TCP/IP ECHO

tcpEchoServer.java

```
import java.net.*;
import java.io.*;
public class tcpEchoServer{
     public static void main(String args[]) throws
IOException
     {
          ServerSocket sock=null;
          BufferedReaderfromClient=null;
          OutputStreamWritertoClient=null;
          Socket client=null;
          try{
               sock=new ServerSocket(4000);
               System.out.println("Server is ready");
               client=sock.accept();
               System.out.println("Client Connected");

               fromClient=new BufferedReader(new
InputStreamReader(client.getInputStream()));

               toClient=new
OutputStreamWriter(client.getOutputStream());

               String line;
               while(true)
               {
                    line= fromClient.readLine();
                    if((line==null)|| line.equals("bye"))
                         break;
                    System.out.println("Client["+line+"]");
                    toClient.write("Server["+line+"]\n");
                    toClient.flush();
```

```
                }
                fromClient.close();
                toClient.close();
                client.close();
                sock.close();
                System.out.println("Client DisConnected");
            }
            catch(IOExceptionioe){
                System.err.println(ioe);
            }
        }
    }
```

Experiment # 11

Object: Program using UDP socket UDP Chat Server/Client.

udpChatServer.java

```
import java.io.*;
import java.net.*;
class udpChatServer
{
      public static int clientport=8040 ,serverport=8050;
      public static void main(String args[ ] ) throws Exception
      {
            DatagramSocketSrvSoc=new DatagramSocket(clientport);
            byte[ ]SData= new byte[1024];
            BufferedReaderbr =new BufferedReader(new
InputStreamReader(System.in));
            System.out.println("Server Ready");
            while(true)
            {
                  byte[ ] RData= new byte[1024];
                  DatagramPacketRPack=new
DatagramPacket(RData,RData.length);
                  SrvSoc.receive(RPack);
                  String Text= new String(RPack.getData());
                  if (Text.trim().length()==0)
                  break;
                  System.out.println("from client<<"+Text);
                  System.out.println("Msg to client:");
                  String srvmsg =br.readLine();
                  InetAddressIPAddr=RPack.getAddress();
                  SData=srvmsg.getBytes();
                  DatagramPacketSPack=new
DatagramPacket(SData,SData.length,IPAddr,serverport);
                  SrvSoc.send(SPack);
            }
            System.out.println("\n Client Quits\n");
            SrvSoc.close();
      }
}
```

udpChatClient.java

```
import java.io.*;
```

```
import java.net.*;
class udpChatClient
{
public static int clientport=8040 ,serverport=8050;
public static void main(String args[ ] ) throws Exception
{
      BufferedReaderbr = new BufferedReader(new InputStreamReader
(System.in));
      DatagramSocketCliSoc = new DatagramSocket(serverport);
      InetAddressIPAddr;
      String Text;
      if(args.length==0)
            IPAddr = InetAddress.getLocalHost();
      else
            IPAddr = InetAddress.getByName(args[0]);
      byte[] SData = new byte[1024];
      System.out.println("Press Enter without text to quit");
      while(true)
      {
            System.out.println("/n Enter text for server:");
            Text = br.readLine();
            SData = Text.getBytes();
            DatagramPacketSPack = new
DatagramPacket(SData,SData.length,IPAddr,clientport);
            CliSoc.send(SPack);
            if(Text.trim().length() == 0)
                  break;
            byte[] RData = new byte[1024];

            DatagramPacketRPack = new DatagramPacket(RData,RData.length);
            CliSoc.receive(RPack);
            String Echo = new String(RPack.getData());
            Echo = Echo.trim();
            System.out.println("From Server<<"+Echo);
      }
      CliSoc.close();
  }
}
```